Customer Churn Prediction & Comparative Machine Learning Analysis Dashboard

Complete End-to-End End-to-End Data Science & Machine Learning Project Guide

Recommended Dataset: Telco Customer Churn Dataset (Kaggle)

This comprehensive project guide is structured to lead students through a full end-to-end Data Science and Machine Learning workflow. Students will explore an industry-relevant business challenge, perform detailed Exploratory Data Analysis (EDA), apply multiple machine learning models, benchmark variations using specific metrics, engineer an interactive dashboard via Streamlit, and deploy the solution to a production cloud environment with a professional GitHub repository strategy.

Phase 1 — Problem Understanding

Objective

The primary technical milestone is to construct a predictive framework capable of forecasting whether a specific service subscriber will terminate their relationship with the enterprise (Churn) or continue their active lifecycle (No Churn).

Business Goal

Acquiring new clients carries a significantly higher financial premium than retaining an active customer base. When customers depart, businesses suffer immediate operational losses and recurring monthly contract revenue slips. By proactively mapping high-risk customer profiles early, marketing teams and customer success managers can introduce precision retention interventions (such as targeted loyalty incentives, specialized service plans, or contract re-negotiations) to stabilize revenue retention.

Phase 2 — Dataset Understanding

Students must analyze and document the architectural properties of the dataset prior to applying processing methodologies. Your documentation must formally outline:

- **Dimensions:** Document total data rows (records) and columns (features).
- **Data Profiles:** Isolate continuous, discrete numerical features and partition categorical labels into nominal vs. ordinal categories. Be sure to check formatting types (e.g., verifying if columns like `TotalCharges` are stored as object strings instead of float numerals).
- **Data Integrity & Noise:** Quantify null parameters, missing variables, and duplicate entities.
- **High-Cardinality and Balance:** Track unique value frequencies across text dimensions and explicitly plot target class imbalances (the precise distribution metric between active accounts and churned users).

Phase 3 — Exploratory Data Analysis (EDA)

Students are required to leverage data visualization engines (**Matplotlib**, **Seaborn**, or **Plotly**) to generate and contextualize the following standard tasks:

1. Univariate Analysis

- **Age & Demographics:** Determine customer distribution parameters across senior citizen splits.
- **Monthly Charges:** Map distribution profiles to track standard customer billing densities.
- **Tenure:** Evaluate structural time distributions representing active relationship lengths.

2. Bivariate Analysis

- **Churn vs. Contract Type:** Evaluate contract behavior characteristics (Month-to-Month, One-year, Two-year).
- **Churn vs. Payment Method:** Investigate if digital automation (electronic checks, paperless billing, auto-pay) acts as a statistical modifier for churn variance.
- **Churn vs. Internet Service Type:** Track structural service delivery issues by studying customer drop-off across Fiber Optic, DSL, or offline components.

3. Correlation Analysis

- **Heatmap Plotting:** Create a comprehensive correlation matrix mapping interactions among numerical features to check for multicollinearity.
- **Feature Importance Précis:** Implement mutual information scoring or tree-base indicators to establish baseline variable impacts.

Phase 4 — Data Preprocessing

Transform raw structured records into sanitized vectors ready for algorithm execution. Ensure consistency between your training splits and validation segments to avoid structural data leakage:

- **Handling Missing Values:** Address null elements appropriately (e.g., imputing standard median vectors, or removing records missing vital information like `TotalCharges`).
- **Categorical Encoding:** Apply **One-Hot Encoding** protocols to nominal multi-class dimensions and utilize **Label/Ordinal Encoding** matrices on logical hierarchy strings.
- **Feature Scaling:** Execute `StandardScaler` or `MinMaxScaler` conversions across wide numerical continuous bounds (e.g., `Tenure` and `MonthlyCharges`) to stabilize distance calculations within KNN or SVM layouts.
- **Train-Test Partitioning:** Allocate records into isolated training and testing matrices using stratified sampling techniques to guarantee matching class targets across divisions.
- **Handling Class Imbalance (Optional):** Apply advanced over-sampling options like **SMOTE** strictly within your training subset boundaries.

Phase 5 — Machine Learning Models

Students **MUST** implement, train, tune, and test at least **six (6) required algorithms** to build a clear comparative framework:

Model Group	Analytical Purpose / Characteristics
1. Logistic Regression	Establishes a solid, interpretable linear classification baseline.
2. Decision Tree	Captures non-linear relationships via intuitive split rules.
3. Random Forest	Leverages bagging ensemble methods to mitigate variance and reduce overfitting.
4. K-Nearest Neighbors (KNN)	Classifies data instances based on distance and local spatial density.
5. Support Vector Machine (SVM)	Optimizes class boundaries using maximum-margin hyperplanes.
6. XGBoost / Gradient Boosting	Applies sequential boosting trees for high-accuracy tabular predictions.

> **Extension Options (Optional):** For advanced credit, you can experiment with and log metrics for extra boosting variants, including *AdaBoost*, *CatBoost*, or *LightGBM*.

Phase 6 — Model Comparison Analysis

This phase is the core analytical component of the assignment. Models must be cross-examined using multiple key classification metrics to identify the best business-driven solution:

Required Metric	Evaluation Priority	Business & Technical Interpretation
Accuracy Score	Overall Balance	Total accurately predicted cases over complete test metrics. Use with caution given target class imbalances.
Precision Score	False Positive Control	Of all accounts classified as churn risks, how many truly left. Higher precision helps manage marketing budget leakage.
Recall Score	False Negative Control	Of all true churned customers, how many did the system identify. Higher recall ensures high-risk accounts are not missed.
F1 Score	Harmonic Compromise	The balanced harmonic mean connecting precision and recall insights into one single metric.
ROC-AUC Score	Separation Efficiency	Measures the classification model's ability to distinguish classes across different probability thresholds.
Confusion Matrix	Error Mapping	Four-quadrant matrix tracking actual vs. predicted classes across True Positives, True Negatives, False Positives, and False Negatives.

Required Comparison Visualization Requirements

Your interactive dashboard interface and notebook files must include the following comparative data graphics:

- Accuracy vs. F1 score comparison bar charts.
- Superimposed multi-model ROC Curves within a single graph interface.
- Side-by-side Confusion Matrix comparison grid.
- Feature Importance tracking charts derived from high-performing ensemble trees.
- Precision-Recall trade-off comparison curves.

Phase 7 — Hyperparameter Tuning

Students must optimize baseline estimators using structured optimization protocols instead of relying on default parameters.

- **Methods:** Deploy cross-validated tuning routines via `GridSearchCV` or `RandomizedSearchCV`.
- **Scope:** Tuning configurations must be applied to at least **Random Forest** and **XGBoost** / **Gradient Boosting** models. Track hyperparameter shifts (e.g., adjusting `n_estimators`, `max_depth`, `learning_rate`) and contrast optimized outputs against initial baselines.

Phase 8 — Streamlit Dashboard Engineering

Students must build a clean, multi-section web application using **Streamlit** to display insights and provide an interactive workspace for users.

Required Web Sections

1. **Home Page:** Framework statement, descriptive high-level project goals, business context overview, and clear asset indexing.
2. **Dataset Explorer:** Dynamic presentation grids featuring searchable data frames, statistical overviews (`df.describe()`), column profiling, and null metrics logs.
3. **EDA Dashboard:** Interactive visual canvases driven by Plotly drop-down selectors where users can explore structural trends, correlation matrices, and demographic variables.
4. **Model Training Section:** An interactive training interface where users select any of the primary models, trigger backend training execution, and view instant performance summaries.
5. **Model Comparison Dashboard:** High-fidelity summary graphics detailing the metrics performance leaderboard alongside an explicit visual callout highlighting the champion production model.
6. **Prediction System Form:** A user-facing query dashboard providing intuitive controls (e.g., numeric input fields, demographic toggle switches, sliders for `Tenure` and `Charges`) that routes inputs into serialized model binaries (`.pkl` pipelines). The web app must display a clear prediction result (**Churn Risk** or **Retained Account**) alongside a specific **Confidence Score Probability Percentage**.

Phase 9 — Production Deployment

Deploy your functional application using public cloud web infrastructure. Recommended hosting environments include:

- **Streamlit Community Cloud:** Deploys applications seamlessly by syncing directly with GitHub source branches.
- **Render:** Alternative deployment platform using dynamic web service runtimes.
- **Hugging Face Spaces:** A reliable hosting platform optimized for application layouts running Streamlit SDK infrastructure.

Phase 10 — GitHub Repository Structure

Students must maintain a structured, logical repository configuration to ensure professional production delivery:

```
project/
│
├── data/              # Data storage folders (raw CSV matrices or source URLs)
├── notebooks/         # Sequenced Jupyter Notebook files covering EDA, Tuning, and Modeling
├── models/            # Serialized production pipeline binaries (.pkl or .joblib assets)
├── visuals/           # Project screenshots, dashboard mockups, and exported chart graphics
│
├── app.py             # Main entry point application file for the Streamlit dashboard
├── utils.py           # Companion scripting files (data extraction, preprocessing helpers)
├── requirements.txt   # Pinpointed runtime libraries and dependency version configurations
└── README.md          # Comprehensive, polished markdown documentation landing page
```

README Documentation Blueprint

The `README.md` profile serves as the principal documentation landing page for the project and must contain: a descriptive title with a functional link to the live deployed app, an overview of the business problem, dataset details, a technologies summary, key EDA discoveries, an evaluation leaderboard table identifying the champion model, and explicit shell scripts outlining local installation and execution parameters (e.g., `pip install -r requirements.txt` followed by `streamlit run app.py`).

Project Evaluation Rubric

Assessment Criteria	Max Marks	Grading Focus Areas & Performance Standards
EDA Quality	**20**	Depth of univariate/bivariate data exploration, analytical depth of insights, visualization clarity, and effective use of Plotly/Seaborn.
Data Preprocessing	**10**	Appropriate handling of missing data, correct categorical encoding strategies, proper scaling application, and complete avoidance of data leakage.
Machine Learning Models	**20**	Successful implementation of all 6 mandatory algorithms, systematic hyperparameter optimization across required frameworks, and proper model serialization.
Model Comparison Analysis	**20**	Thorough generation of key classification metrics, accurate technical interpretation of results, and strong data-backed justification for the champion model selection.
Streamlit Dashboard	**20**	Polished UI/UX design, complete interactive functionality across all 6 sections, a reliable web prediction form, and error-free execution.
Deployment & Repository	**10**	Fully functional public deployment URL, clean and well-structured project folders, and an extensively documented professional README.md file.
TOTAL EVALUATION	**100 Marks**	**Comprehensive evaluation metrics across an end-to-end Machine Learning pipeline.**

Final Submission Checklist

- **Jupyter Notebook:** Completely executed without execution errors, containing detailed inline markdown commentary.
- **Streamlit Scripts:** Fully functional, production-ready code files (`app.py` and `utils.py`) using relative paths.
- **Requirements File:** A validated, version-locked `requirements.txt` file to build the cloud deployment environment.
- **Production Environment:** A live, accessible application link hosted on Streamlit Cloud, Render, or Hugging Face.

- **GitHub Repository:** A well-organized public repository with a professional commit history, clear directory structure, and an informative, image-supported `README.md`.